

作业发布时间：2025/04/10 星期四

本次作业要求如下：

1.截止日期：2025/04/20 周日晚 24:00

2.提交作业给本课程 QQ 群里的助教

3.命名格式：附件和邮件命名统一为“第一次作业+学号+姓名”，作业为 PDF 格式；

4.注意：

(1) 选择、判断和填空只需要写答案，大题要求有详细过程，过程算分。

(2) 答案请用另一种颜色的笔回答，便于批改，否则视为无效答案。

(3) 大题的过程最好在纸上写了拍照，放到 word 里。

(4) 本次作业由 Part1 和 Part2 两部分，需要全部作答

5.本次作业遇到问题请联系课程群里的助教。

Part1 X86 汇编

1. C 语言程序中的整数常量、整数常量表达式是在 (D) 阶段初始化和计算的。

(A) 预处理 (B) 执行 (C) 链接 (D) 编译

2. 关于 Intel 的现代 X86-64 CPU 正确的是 (C)

A. 属于 RISC B. 属于 MISC C. 属于 CISC D. 属于 NISC

3. 下列叙述正确的是 (D)

A. X86-64 指令 "mov \$-1, %eax" 会使 %rax 的值变成 0xffffffffffffffff

B. 在一条指令执行期间，CPU 不会两次访问内存

C. CPU 不总是执行 CS::RIP 所指向的指令，例如遇到 call、ret 指令时

D. 一条 mov 指令不可以使用两个内存地址操作数

4. 在 x86-64 系统中，调用函数 int gt (long x, long y) 时，保存参数 y 的寄存器是 (B)

A. %rdi B. %rsi C. %rax D. %rdx

5. 在 x86-64 系统中，调用函数 long gt (long x, long y) 时，保存返回值的寄存器是 (C)

A. %rdi B. %rsi C. %rax D. %rbx

6. 下列传送指令中，哪一条是正确的 (D)

A. movb \$0x105, (%ebx)

B. movl %rdi, %rax

C. movq %rdi, \$105

D. movl 4(%rsp), %eax

7. C 语言程序定义了结构体 struct noname {char c; long n; int k; float *p; short a;}; 若该程序编译成 64 位可执行程序，则 sizeof(noname) 的值是 40。

8. 在 X86-64 中，程序的栈存放在内存中，栈顶元素的地址使所有栈中元素中最小的，栈指针寄存器 %rsp 保存着栈顶元素的地址。

9. While 循环的两种展开方法分别是什么：(条件判断跳转式) (条件反转跳转式)

下面代码是哪种？（条件反转跳转式）

```
long fact_while_jm_goto(long n)
{
    long result = 1;
    goto test;
loop:
    result *= n;
    n = n-1;
test:
    if (n > 1)
        goto loop;
    return result;
}
```

10. 已知内存和寄存器中的数值情况如下：

内存地址	值	寄存器	值
0x100	0xff	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10c	0x11		

请填写下表，给出对应操作数的值：

操作数	值
%rax	0x100
(%rax)	0xff
9(%rax,%rdx)	0x11
0xfc(,%rcx,4)	0xff
(%rax,%rdx,4)	0x11

- 1. % rax: 寄存器 % rax 的值为 0x100。
- 2. (% rax): 指存储在 % rax 中的地址所对应的内存值，为 0x100。地址 0x100 包含 0xff。
- 3. 9 (% rax,% rdx): $\% \text{ rax} + \% \text{ rdx} + 9 = 0x100 + 0x3 + 0x9 = 0x10C$ 。地址 0x10C 包含 0x11。
- 4. 0xfc (,% rcx,4): $0xfc + \% \text{ rcx} * 4 = 0xfc + 0x1 * 4 = 0xfc + 0x4 = 0x100$ 。地址 0x100 包含 0xff。
- 5. (% rax,% rdx,4): $\% \text{ rax} + \% \text{ rdx} * 4 = 0x100 + 0x3 * 4 = 0x100 + 0xC = 0x10C$ 。地址 0x10C 包含 0x11。

11. 有下列 C 函数：

```
long max(long x, long y)
{
    long result;
    if(x >= y) {
```

```

        result = x;
    }else{
        result = y;
    }
    return result;
}

```

请写出红色部分代码使用条件数据传输来实现条件分支的等价形式,并给出对应的条件传送指令(初始执行指令 `movq %rdi, %rax`)。

答:

```

movq    %rdi, %rax          # 把 x 将放到返回值寄存器%rax 中
cmpq    %rsi, %rdi          # 比较 x (rdi) 和 y (rsi), 相当于 x - y
cmovl    %rsi, %rax         # 如果 x < y (即 x 不是最大), 用 y 替换%rax

```

12. 阅读的 sum 函数反汇编结果, 解释 1-5 每行指令的功能和作用

```

4004e7 <sum>:
push %rbp #①
mov %rsp,%rbp #②
movl %rdi,-0x4(%rbp) #③
jmp 400512 <sum+0x2b>
4004f4:
mov -0x4(%rbp),%eax
cltq
mov 0x601030(,%rax,4),%edx
mov 0x200b3e(%rip),%eax #601044 <val>
add %edx,%eax
mov %eax,0x200b36(%rip) #601044 <val>
addl $0x1,-0x4(%rbp)
400512:
cmpl $0x3,-0x4(%rbp)#④
jl 4004f4 <sum+0xd>#⑤
mov 0x200b26(%rip),%eax # 601044 <val>
pop %rbp
Retq

```

答:

1. `push %rbp`: 保存调用者的基址指针, 将当前的 `%rbp` 压入栈中, 以便在函数结束时可以恢复它。
2. `mov %rsp, %rbp`: 将当前栈指针 `%rsp` 的值赋给基址指针 `%rbp`。

3. `movl %rdi, -0x4(%rbp)`: 将传入的参数保存到栈上的局部变量 `-0x4(%rbp)` 处。
4. `cmpl $0x3, -0x4(%rbp)`: 比较局部变量 `-0x4(%rbp)` 与常数 3。
5. `j1 4004f4 <sum+0xd>`: 若前面的比较结果是小于, 即 `-0x4(%rbp) < 3`, 则跳转到地址 4004f4, 继续循环体。

13. 假设变量 `sp` 和 `dp` 被声明为类型:

```
Src_t *sp;
```

```
Dest_t *dp;
```

这里的 `Src_t` 和 `Dest_t` 是用 `typedef` 声明的数据类型, 我们想使用适当的数据传送指令来实现下面的操作:

```
*dp = (Dest_t) *sp;
```

`sp` 和 `dp` 的值分别存储在 `%rdi` 和 `%rsi` 中, 对下表的每个表项, 请写出合适的两条传送指令 (如需用到其他寄存器, 使用 `%rax`)。注意: 规定如果强制类型转换既涉及大小变化又涉及符号变化时, 操作应先改变大小。

Src_t	Dest_t	指令
char	int	<u>①</u> <code>movsbl (%rdi), %eax</code> <code>movl %eax, (%rsi)</code>
char	unsigned	<u>②</u> <code>movsbl (%rdi), %eax</code> <code>movl %eax, (%rsi)</code>
unsigned char	long	<u>③</u> <code>movzbl (%rdi), %eax</code> <code>movq %rax, (%rsi)</code>
int	char	<u>④</u> <code>movl (%rdi), %eax</code> <code>movb %al, (%rsi)</code>
unsigned	unsigned char	<u>⑤</u> <code>movl (%rdi), %eax</code> <code>movb %al, (%rsi)</code>

答:

1. `movsbl (%rdi), %eax` # 从 `[sp]` 取 1 字节符号扩展到 `eax` (32 位)
 `movl %eax, (%rsi)` # 将结果写入 `[dp]`
2. `movsbl (%rdi), %eax` # 先符号扩展为 32 位
 `movl %eax, (%rsi)` # 再写入 `[dp]`
3. `movzbl (%rdi), %eax` # 零扩展为 32 位
 `movq %rax, (%rsi)` # 写入 `[dp]`, 因为目标是 8 字节
4. `movl (%rdi), %eax` # 取 4 字节 `int`
 `movb %al, (%rsi)` # 写入低位 1 字节到 `[dp]`
5. `movl (%rdi), %eax` # 取 4 字节 `unsigned`
 `movb %al, (%rsi)` # 写入低位 1 字节到 `[dp]`

14. 设有 C 代码如下:

```
typedef struct {
```

```

    int a[2];
    double d;
} struct_t;
double fun(int i) {
    volatile struct_t s;
    s.d = 3.14;
    s.a[i] = 1073741824; /* Possibly out of bounds */
    return s.d;
}

```

其运行时的栈结构如下图所示：

临界状态	6
?4 字节	5
?4 字节	4
d7 ... d4	3
d3 ... d0	2
a[1]	1
a[0]	0

请分别求出：调用 fun (x) 的结果，其中 x 分别为 0，1，2，3，4，6 并给出分析过程。

答：

i = 0：写入 a[0] 的位置，合法，不影响 d → 返回值：3.14
 i = 1：写入 a[1] 的位置，合法，不影响 d → 返回值：3.14
 i = 2：写入偏移 8，即覆盖 d 的低 4 字节，破坏了 d → 返回值：≈3.14
 i = 3：写入偏移 3，即覆盖 d 的高 4 字节，d 整体被破坏 → 返回值：不是 3.14，值被破坏，变为 2.000001
 i = 4：写入偏移 4，可能覆盖栈中其他局部变量或保留区，不影响 s.d，但存在潜在风险 → 返回值：3.14（一般）
 i = 6：写入偏移 6，覆盖可能是返回地址或更关键数据，程序行为不可预测，甚至可能崩溃 → 返回值：3.14（可能异常终止）

15. 简述缓冲区溢出攻击的原理以及防范方法（2 种）

答：

原理：程序中的缓冲区通常有固定的大小。当数据超出缓冲区时，它会覆盖内存中的其他内容（如函数返回地址、控制信息等）。攻击者通过构造特定的输入数据，使溢出的数据覆盖程序的控制信息（如返回地址），并将其替换为恶意代码的地址。当程序执行到该位置时，会跳转到攻击者指定的恶意代码，从而实现代码注入攻击。

防范方法：

1. 使用栈保护：通过在栈帧中插入一个 canary 值来实现。该值会在函数调用时设置，并且在函数返回时进行验证。如果攻击者通过缓冲区溢出覆盖了这个 canary 值，栈保护机制会发现异常，并触发程序异常终止，防止恶意代码的执行。
2. 地址空间布局随机化：通过随机化程序和库的内存地址，使得攻击者无法预测目标地址的具体位置。即使攻击者能够利用缓冲区溢出，无法准确地预测覆盖返回地址等内存位置，从而防止攻击者控制程序流。

Part2 处理器体系结构

1. Y86-64 的指令 ret 编码长度为（A ）。
A. 1 字节 B. 2 字节 C. 9 字节 D. 10 字节
2. Y86-64 的 CPU 顺序结构设计与实现中，分成（A ）个阶段
A. 5 B. 6 C. 7 D. 8
3. Y86-64 的 CPU 流水线结构设计与实现中，分成（A ）个阶段
A. 5 B. 6 C. 7 D. 8

判断题：

4. Y86-64 的顺序结构实现中，寄存器文件读时是作为时序逻辑器件看待（错误）

5. 现代超标量 CPU 指令的平均周期接近于 1 个但大于 1 个时钟周期（正确）

6. 下表是 CPU 的某个场景，解释：加载指令（`lrmovq` 和 `popq`）占有所有执行指令的 20%，其中 15%会导致 加载/使用 冒险。条件分支指令占有所有执行指令的 25%，其中 40%不选择分支。返回指令占有所有执行指令的 3%。完成下表：

原因	名称	指令频率	条件频率	气泡数	总处罚	CPI
加载使用	lp	0.20	0.15	1	0.32	1.32
预测错误	mp	0.25	0.40	2		
返回	rp	0.03	1.00	3		

7. 在 Y86-64 架构的机器上，有下列汇编代码，请指出 jne t 指令之后执行的那条指令的地址为 0x00b （顺序执行，不考虑流水线）。

```

0x000:    xorq %rax,%rax
0x002:    jne t
...
0x019: t:  irmovq $3, %rdx
0x023:    irmovq $4, %rcx
0x02d:    irmovq $5, %rdx

```

8. 请写出 Y86-64 的 CPU 流水线结构设计与实现中各流水线阶段的名称（注意顺序不要错位）。

答：

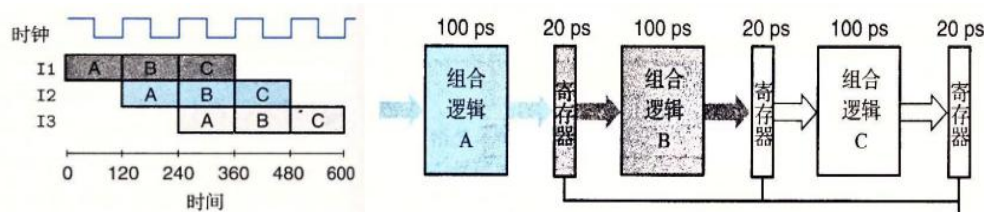
取指、译码、执行、访存、写回。

9. Y86-64 流水线 CPU 中的冒险的种类与处理方法。

答：

1. 数据冒险，处理方式：暂停、转发、插入气泡；
2. 控制冒险，处理方式：暂停、分支预测、延迟分支、刷新流水线；
3. 结构冒险，处理方式：为不同流水线阶段提供独立的硬件资源、暂停。

10. 假设有一个理想的三阶段流水线，执行指令为 I1（Instruction1），I2，I3，其时序图与电路示意图如图所示：（P321）



请分析不同时间点各寄存器中保存值所属指令，并完成下表（填入 I1，I2，I3，None。寄存器 1 表示左数第一个寄存器）

	寄存器 1	寄存器 2	寄存器 3
239	I1	None	None
241	I2	I1	None
300	I2	I1	None
361	I3	I2	I1

t = 239:

寄存器 1：在时钟沿 120 ps 时，I1 完成阶段 A，其结果被锁存进寄存器 1。到 240 ps 时钟沿之前，寄存器 1 一直保存 I1 的结果。寄存器 2 & 3：在时

钟沿 120 ps 时，没有指令完成阶段 B 或 C，所以寄存器 2 和 3 为空 (None)。

t = 241:

时钟沿发生在 240 ps。

寄存器 1: I2 在 240 ps 完成阶段 A，其结果被锁存进寄存器 1。

寄存器 2: I1 在 240 ps 完成阶段 B，其结果被锁存进寄存器 2。

寄存器 3: 没有指令在 240 ps 完成阶段 C，寄存器 3 仍为空 (None)。

t = 300:

时间点在 240 ps 和 360 ps 时钟沿之间。寄存器内容保持 240 ps 时钟沿锁存的状态。

寄存器 1: I2

寄存器 2: I1

寄存器 3: None

t = 361:

时钟沿发生在 360 ps。

寄存器 1: I3 在 360 ps 完成阶段 A，其结果被锁存进寄存器 1。

寄存器 2: I2 在 360 ps 完成阶段 B，其结果被锁存进寄存器 2。

寄存器 3: I1 在 360 ps 完成阶段 C，其结果被锁存进寄存器 3。

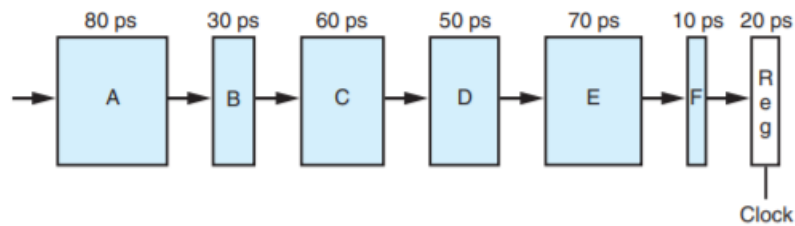
11. 请根据描述在 Y86-64 顺序实现的条件下完成下面表格。

(1) 请写出 Y86-64 顺序实现的 push rA 指令在各阶段的微操作。

(2) 请写出 Y86-64 顺序实现的 pop rA 指令在各阶段的微操作。

指令	push rA	pop rA
取指	$f_pc \leftarrow PC$	$f_pc \leftarrow PC$
	$icode:ifun \leftarrow M1[f_pc]$	$icode:ifun \leftarrow M1[f_pc]$
	$rA:rB \leftarrow M1[f_pc+1]$	$rA:rB \leftarrow M1[f_pc+1]$
	$valP \leftarrow f_pc + 2$	$valP \leftarrow f_pc + 2$
译码	$valA \leftarrow R[rA]$	$valB \leftarrow R[RSP]$
	$valB \leftarrow R[RSP]$	$addr \leftarrow valB$
执行	$valE \leftarrow valB - 8$	$valE \leftarrow valB + 8$
访存	$M8[valE] \leftarrow valA$	$valM \leftarrow M8[addr]$
写回	$R[RSP] \leftarrow valE$	$R[rA] \leftarrow valM, R[RSP] \leftarrow valE$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow valP$

12. 假设我们分析图中的组合逻辑，认为它可以分为 6 个块，以此命名为 A、B、C、D、E、F，延迟分别为 80，30，60，50，70，10（单位 ps），如下图所示。



在这些块之间插入流水线寄存器，就得到这一设计的流水线化的版本。根据在哪里插入流水线寄存器，会出现不同的流水线深度(有多少个阶段)和最大吞吐量的组合。假设每个流水线寄存器的延迟为 20ps。

请根据以上描述，回答下列问题：

(1) 只插入一个寄存器，得到一个两阶段的流水线。要使吞吐量最大化，该在哪里插入寄存器呢？吞吐量和延迟是多少？

若插入一个寄存器，得到一个两阶段的流水线。为了使吞吐量最大化，应将寄存器插入在模块 C 和 D 之间。此时两段的组合逻辑延迟分别为 $A+B+C = 80+30+60 = 170$ ps 和 $D+E+F = 50+70+10 = 130$ ps，最大段延迟为 170 ps，加上寄存器延迟 20 ps，得到的时钟周期为 190 ps，对应最大吞吐量为 $1/190$ ps。

(2) 要使一个三阶段的流水线的吞吐量最大化，该将两个寄存器插在哪里呢？吞吐量和延迟是多少？

若插入两个寄存器，得到一个三阶段的流水线。为了使吞吐量最大化，应将寄存器分别插入在模块 B 和 D 之后，划分为 $A+B$ 、 $C+D$ 、 $E+F$ 三段，延迟分别为 110 ps、110 ps 和 80 ps。最大段延迟为 110 ps，加上寄存器延迟 20 ps，得到的时钟周期为 130 ps，对应最大吞吐量为 $1/130$ ps。

(3) 要使一个四阶段的流水线的吞吐量最大化，该将三个寄存器插在哪里呢？吞吐量和延迟是多少？

若插入三个寄存器，得到一个四阶段的流水线。为了使吞吐量最大化，应将寄存器插入在模块 A、C 和 D 后，划分为 A、 $B+C$ 、D、 $E+F$ 四段，延迟分别为 80 ps、90 ps、50 ps 和 80 ps。最大段延迟为 90 ps，加上寄存器延迟 20 ps，得到的时钟周期为 110 ps，对应最大吞吐量为 $1/110$ ps。

(4) 要得到一个吞吐量最大的设计，至少要有几个阶段？描述这个设计及其吞吐量和延迟。

若要实现最大吞吐量的设计，至少需要六阶段流水线，即每个模块后都插入一个寄存器（共五个）并加上原有输出寄存器。各段组合逻辑延迟分别为 80 ps、30 ps、60 ps、50 ps、70 ps、10 ps。最大段延迟为 80 ps，加上寄存器延迟 20 ps，时钟周期为 100 ps，对应吞吐量为 $1/100$ ps。该设计的总延迟为所有逻辑延迟和加上 6 个寄存器延迟，即 300 ps + 6×20 ps = 420 ps。

13. 下面是一个程序段，请根据 Y86-64 的微指令和流水线数据相关的知识，试解释为什么在 call 指令之前要插入 3 个 nop 指令。

```
0x000:    irmovq Stack,%rsp    # Intialize stack pointer
```

```
0x00a:    nop
0x00b:    nop
0x00c:    nop
0x00d:    call p                # Procedure call
0x016:    irmovq $5,%rsi          # Return point
0x020:    halt
```

答：

在 `irmovq Stack, %rsp` 和 `call p` 指令之间插入 3 个 `nop` 指令，是为了解决数据冒险。`call` 指令依赖于 `%rsp` 寄存器的值，而这个值是由前一条 `irmovq` 指令写入的。由于流水线的特性，`irmovq` 的写回操作发生在写回阶段，而 `call` 需要在译码或执行阶段就用到这个值。为了确保 `call` 指令能读取到 `irmovq` 写入的最新值，需要插入足够的 `nop` 指令（流水线气泡/暂停）来延迟 `call` 指令的执行，直到 `%rsp` 的值准备好。这 3 个 `nop` 提供了必要的延迟。